Assignment 4 Report: Neural Style Transfer

Introduction:

In this assignment, I implemented neural style transfer using a pretrained VGG19 network in PyTorch. The goal was to generate a new image that keeps the content of one image while applying the artistic style of another image. This method is based on the paper by Gatys et al.

The idea is that a convolutional neural network can separate content and style. Deeper layers capture the structure of the image (like objects and layout), while earlier layers capture textures, colors, and patterns. By combining these, we can create a stylized image.

Implementation:

I used the pretrained VGG19 model and only used its feature extraction layers. I froze all the model parameters since I was not training the network. Instead, I optimized the pixels of a target image.

Content Representation:

The content of the image was extracted from the layer called `conv4_2`. This layer captures the structure of the image, such as shapes and layout. The content loss was calculated as the mean squared difference between the features of the target image and the content image.

Style Representation:

The style was extracted using multiple layers:

- conv1_1
- conv2_1
- conv3_1
- conv4_1
- conv5_1

To represent style, I used something called a Gram matrix, which captures how different features are related to each other. This helps capture textures and patterns instead of exact positions in the image.

Total Loss:

The total loss is a combination of content loss and style loss. The content weight controls how much structure is preserved, and the style weight controls how strong the artistic effect is.

In my implementation:

- content weight = 1
- style weight = 1e7

The target image was initialized as a copy of the content image and optimized using the Adam optimizer

Important Code Snippet:

```
print("Learning rate:", 0.01)

for ii in range(1, steps + 1):

    # get target image features
    target_features = get_features(target, vgg)

    # content loss
    content_loss = torch.mean((target_features['conv4_2'] - content_features['conv4_2']) ** 2)

    # style loss
    style_loss = 0
    for layer in style_weights:
        target_feature = target_features[layer]
        _, d, h, w = target_feature.shape

        # target gram matrix
        target_gram = gram_matrix(target_feature)

        # style gram matrix
        style_gram = style_grams[layer]

        # layer style loss
        layer_style_loss = style_weights[layer] * torch.mean((target_gram - style_gram) ** 2)

        style_loss += layer_style_loss / (d * h * w)

    # total loss
    total_loss = content_weight * content_loss + style_weight * style_loss
```

Experiments and Observations:

I tested different settings to see how they affect the result.

Experiment 1 (Baseline)

- Steps: 2000
- Style weight: 1e6

Observation:
The output kept the structure of the original image, but the style effect was weak and looked slightly noisy.

Experiment 2 (Higher Style Weight)

- Steps: 3000
- Style weight: 1e7

Observation:

The style became much stronger. The image had more visible textures and looked more like a painting. The sky and lighting areas showed stronger artistic patterns.

Experiment 3 (More Iterations)

Increasing the number of steps improved the quality of the image. The textures became smoother and more consistent over time

Results

The final output successfully combined the house image with the Van Gogh style. The structure of the building was preserved, while the colors and textures were transformed into a painted look.

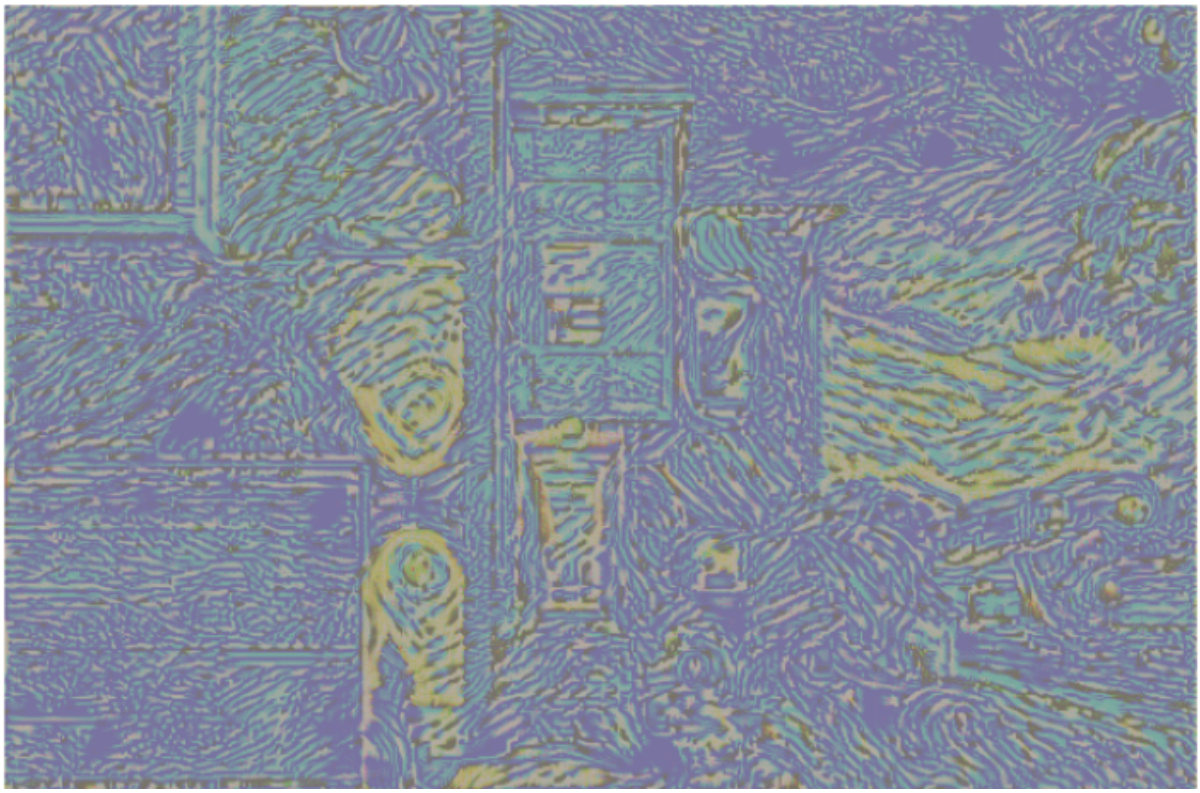
The result was not perfectly smooth like the original paper, but it clearly demonstrated how neural style transfer works.

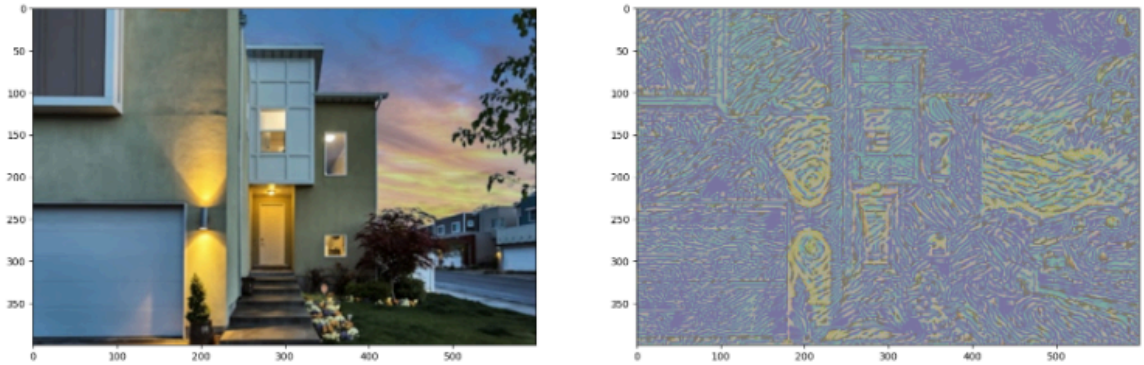
Conclusion

In this assignment, I implemented neural style transfer and generated stylized images using a pretrained model. I learned how different layers in a CNN represent different types of information and how style can be captured using feature correlations.

I also observed how changing hyperparameters like style weight and number of steps affects the output. This helped me understand how neural networks can be used for image generation, not just classification.

Images :



```python
# display content and final, target image
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(20, 10))
ax1.imshow(im_convert(content))
ax2.imshow(im_convert(target))
```

```
<matplotlib.image.AxesImage at 0x7c45ec87b500>
```